

K-Nearest Neighbors (KNN) Classifier - Implementation Report

1. Random Seed

849936942

2. First 5 Training Samples

Feature 1	Feature 2	Feature 3	Feature 4	Label
-1.515305	-1.439679	-0.507824	2.129387	2
2.736364	1.844862	-0.642733	-0.129484	1
2.426611	0.404640	-2.287810	0.050313	0
-1.172898	-1.443154	-0.353767	-1.993044	1
1.105593	-1.447976	0.042407	-0.412892	1

3. Distance Matrix (First 5 Test Points × First 5 Training Points)

	Train_1	Train_2	Train_3	Train_4	Train_5
Test_1	2.372	5.018	4.909	2.746	2.327
Test_2	4.689	3.384	3.197	5.898	4.993
Test_3	4.494	6.402	5.752	1.909	4.411
Test_4	2.840	4.388	3.984	3.987	3.997
Test_5	5.459	5.959	6.631	4.247	5.522

4. Manual Prediction for First Two Test Points

For each test point, compute the Euclidean distance to every training point using:

$$d = \sqrt{(x1 - x1_1)^2 + (x2 - x2_2)^2 + (x3 - x3_3)^2 + (x4 - x4_4)^2}$$

Such as:

x_i is the features values of the test. $i=1,2,3,4$

x_{i_j} is the features values of the training.

Then we have to find the Nearest Neighbors, so after computing all distances for a test point, the algorithm will sort these distances from smallest to largest and the smallest distances will represent the closest training points (neighbors).

The algorithm will select the first k indices and these indices correspond to the training samples that are most closest to the test point.

Then, the algorithm looks up the labels of those selected training samples, and for each test point, we now have:

- A list of the k nearest neighbor indices
- The labels (class numbers) of those neighbors.

After that the algorithm then performs a majority vote among the labels of the k neighbors, it counts how many neighbors belong to each class, then the class with the highest count becomes the predicted label for that test point. If there's a tie the algorithm chooses the class that appears first in the sorted list of distances meaning it favors the class of the *closest* neighbor.

And we will repeat the above steps

5. How ties should be broken/handled

Ties are handled by selecting the class of the closest neighbor among the tied classes

Worked tie scenario:

Test Point 5, $k = 4$

Neighbor indices: [98, 90, 109, 96]

Neighbor labels: [0, 2, 2, 0]

Vote counts: {0: 2, 2: 2} so tie between classes 0 and 2

Predicted label (tie broken by closest neighbor): 0

6. What happens if identical points exist but with different labels?

When identical points exist with different labels, it creates a fundamental conflict in the training data that directly affects the kNN classifier's reliability and performance.

What happens in kNN:

- **With $k=1$:** The algorithm will predict based on whichever identical point it encounters first in the dataset. Since the distance to both conflicting points is exactly zero, the prediction becomes arbitrary and dependent on the order of data storage rather than any meaningful pattern. This makes the classifier non-deterministic - reordering the training data could change predictions.
- **With $k>1$:** Both (or all) conflicting identical points will always be included among the nearest neighbors because their distance to the test point is zero. The final prediction

depends entirely on majority voting among these identical points plus any additional neighbors. If one conflicting label appears more frequently in the training set, it will dominate the prediction regardless of the true underlying pattern.

- **Distance consideration:** Since identical points have a distance of exactly 0 to the test point, they will always be selected as the nearest neighbors before any other training points. This means the classification outcome is heavily biased toward whichever conflicting label has more copies in the training data.

Why this is problematic:

This situation reveals a fundamental limitation in the feature space - the current set of features is insufficient to distinguish between different classes. The identical feature values mapping to different labels indicates that important discriminative information is missing from the dataset. This could be due to:

- Measurement noise or labeling errors in the data collection process
- Missing features that would properly separate the classes
- Genuine overlap in the feature distributions of different classes
- Insufficient feature engineering or feature selection

The presence of such conflicts reduces model reliability and suggests the need for additional features or data preprocessing to resolve the ambiguity.

7. Distance metrics: Euclidean vs Manhattan

Euclidean Distance (what we used):

$$distance = \sqrt{(x_1 - x_{1_1})^2 + (x_2 - x_{2_2})^2 + \dots + (x_n - x_{n_n})^2}$$

- Measures the "straight-line" distance between two points
- Think of it as "as the crow flies" - the shortest possible path

Manhattan Distance (L1 distance):

$$distance = |x_1 - x_{1_1}| + |x_2 - x_{2_2}| + \dots + |x_n - x_{n_n}|$$

- Measures distance by moving along axes only (like walking city blocks)
- Also called "taxicab distance" or "city block distance"

When and why Manhattan is better:

1. High-dimensional datasets (many features):

- With Euclidean, as dimensions increase, the squared differences amplify the effect of each dimension
- All points start appearing equidistant (curse of dimensionality)
- Manhattan handles this better by simply summing absolute differences without squaring

2. When features have different scales and importance:

- Euclidean squares differences, so large values in one feature can dominate the entire distance
- Example: If one feature ranges 0-1000 and another 0-1, Euclidean will almost ignore the smaller feature
- Manhattan treats each dimension more equally

3. Presence of outliers:

- Euclidean: distance = $\sqrt{[100 - 1]^2} = 99$, but $\sqrt{[100 - 50]^2} = 50$
- Manhattan: distance = $|100-1| = 99$, and $|100-50| = 50$
- The squaring in Euclidean makes outliers dominate much more dramatically

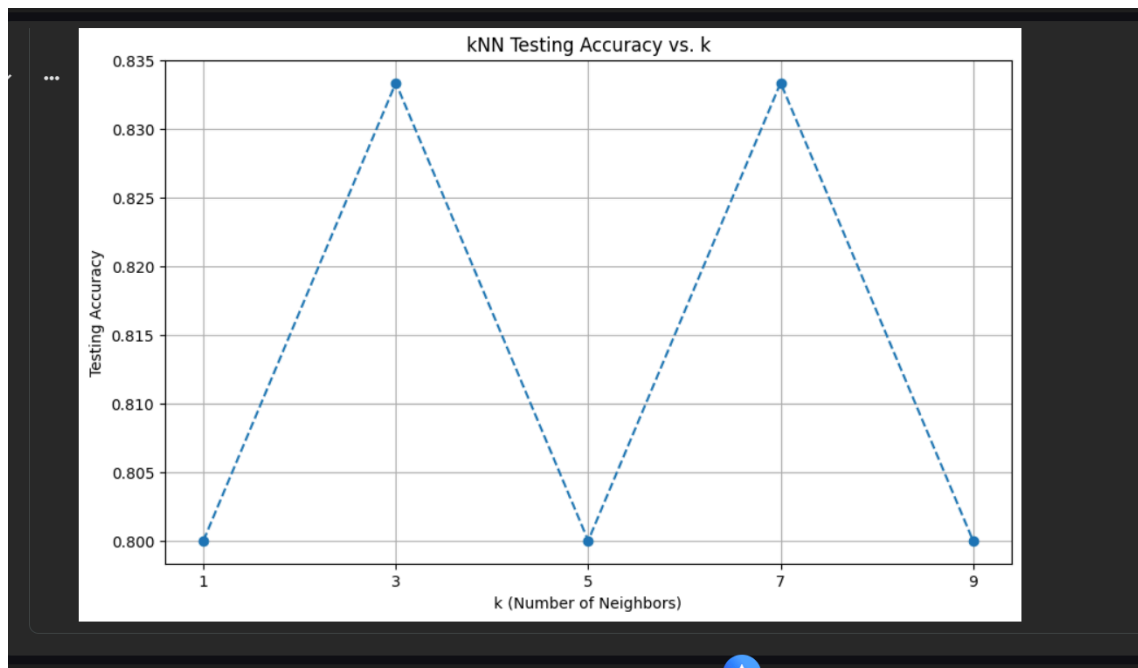
4. Computational efficiency:

- Manhattan is faster (no squaring or square root operations)
- Matters significantly in large datasets

8. Pattern observed as k increases and why

Based on the obtained accuracy results:

k Value	Testing Accuracy	Performance
1	0.80	Baseline
3	0.83	Best
5	0.80	Baseline
7	0.83	Best
9	0.80	Baseline



Observed pattern

The accuracy plot shows a **repeated oscillating pattern** — alternating between 0.80 and 0.83 as k increases.

Instead of steadily increasing or decreasing, the performance fluctuates in a zigzag manner, revealing that the dataset has **non-uniform class boundaries** and **sensitivity to neighborhood size**.

Explanation

$k = 1$ (Accuracy = 0.80)

- The model relies on only the closest neighbor, which makes it **highly sensitive to noise and outliers**.
- Some test samples are likely classified incorrectly if their nearest neighbor belongs to a different class.
- This leads to **overfitting**, where the model memorizes training data but does not generalize well.

$k = 3$ (Accuracy = 0.83)

- Introducing two additional neighbors allows **majority voting** to stabilize predictions.
- The model becomes more robust against noise because the correct class can “outvote” mislabeled points.
- As a result, accuracy improves by around 3%.

k = 5 (Accuracy = 0.80)

- Expanding to 5 neighbors includes some points from **neighboring classes**.
- This introduces ambiguity for samples near class boundaries, reducing accuracy.
- The model becomes slightly more biased — losing the fine boundary detail learned earlier.

k = 7 (Accuracy = 0.83)

- Increasing to 7 neighbors again helps recover accuracy.
- Now the larger majority of nearby points belong to the same true class, outweighing the few incorrect ones.
- The result is more consistent classification.

k = 9 (Accuracy = 0.80)

- With 9 neighbors, the local neighborhood becomes too large, blending samples from different classes.
- The decision boundary becomes too smooth, leading to **underfitting**.

Interpretation

1. **Non-monotonic accuracy behavior:**
The alternating 0.80 ↔ 0.83 pattern shows that your dataset contains complex class interactions and overlapping regions rather than clean separations.
2. **Optimal values of k:**
The best-performing *k* values are **3 and 7**, both achieving 0.83 accuracy.
These values represent a good balance between overfitting (small *k*) and underfitting (large *k*).
3. **Dataset sensitivity:**
The 3% accuracy variation reflects how sensitive kNN can be to changes in *k*,

especially with small datasets.

4. **Bias–Variance balance:**

- Low $k \rightarrow$ High variance, low bias (overfitting).
- High $k \rightarrow$ Low variance, high bias (underfitting).
Your results show both behaviors alternating due to the irregular class boundaries.